From Mondrian to Escher: a tiling language interpreter

Second project of Languages and Programming Environments 25/26

8 May 2026

Due date: 24 May 2026

v1.1 – 9 May 2026: added figures and corrected typos

v1.2 – 18 May 2026: clarified description of the `repeatplanar` instruction

1 Background

This story begins with a visit to two Netherlands' museums, Escher's museum in The Hague and the Mondrian's birthplace in Amersfoort. Both artists are known for their unique styles and contributions to the art world. Escher is famous for his intricate and mind-bending tessellations, while Mondrian is renowned for his abstract geometric compositions.

2 Problem Statement

The problem presented here is to develop an interpreter for a simple tiling language inspired by Escher's and Mondrian's artworks. The starting point is an interpreter for a simple language that draws basic geometric shapes, such as squares and lines. The task is to understand a given interpreter that implements basic operations and extend it with more operations to support more complex tiling patterns, allowing users to create their own Escher-like tessellations and Mondrian-inspired compositions.

The given interpreter is capable of interpreting a simple program like the following:

```
# Simple tiling program

main:
square red 0 0 50 50
square blue 50 50 50 50
line black 2 0 0 100 100
```

```
text black 20 10 20 "Hello, Escher!"
image ../escher.png 20 20 1 0
```

And to produce a simple tiling pattern with two squares, one red and one blue, positioned at different locations on the canvas and a line over the squares. The `main` label serves as the entry point of the program, and the `square` instructions define the properties of each square, such as colour, fill type, position, and size. The `line` instruction defines a line with specific colour, width, and coordinates. The `text` and `image` instructions add text and image elements to the composition. The visible result is:

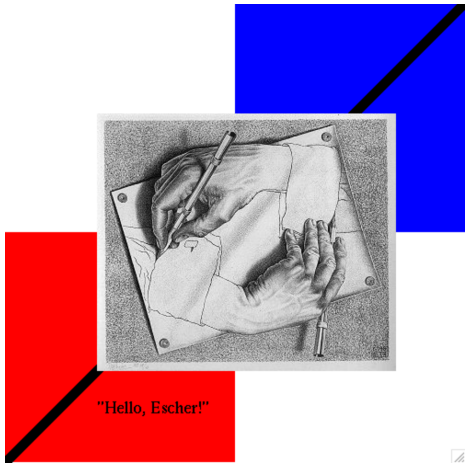

Figure 1: Simple Composition

The operations to be added as part of the extension are `call`, that executes the instructions defined by another label, `translate` that shifts the whole referential system by a certain amount in the `x` and `y` directions, `rotate` that rotates the whole referential system by a certain angle, `repeatplanar`, and `repeatrotate` that rotates a given shape a given amount of degrees a given number of times. These operations will enable users to create more intricate and visually appealing patterns by manipulating the position, orientation, and repetition of shapes.

Take the example of "Composition II in Red, Blue, and Yellow" by Mondrian depicted below.

This painting can be represented in the tiling language as follows:

```
# Mondrian painting

main:
call squares
call lines
```

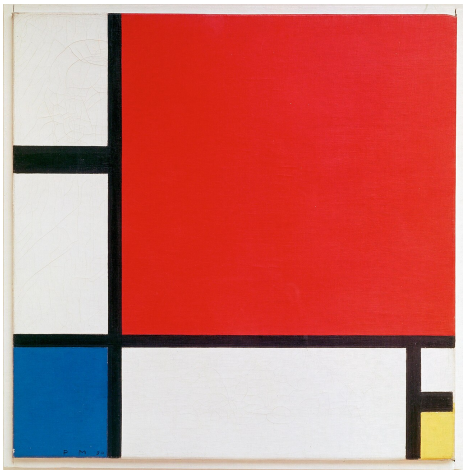

Figure 2: Representation of Mondrian's Composition II in Red, Blue, and Yellow

```
squares:
square blue 0 0 20 20
square red 22 22 78 78
square yellow 93 0 7 10

lines:
line black 2 21 0 21 100
line black 2 0 21 100 21
line black 4 0 50 20 50
line black 2 93 0 93 20
line black 2 93 10 100 10
```

In the picture above, `#` is a line comment marker, `main` is a label and the entry point of the program. A label defines a figure abstraction defined by the instructions until the word `end`. The instruction `call` is used to invoke another figure. The `squares` label defines three squares with different colours and positions, while the `lines` label defines several lines that create the grid-like structure of the painting.

The visible result is therefore:

2.1 Instruction set

The interpreter supports the following instructions:

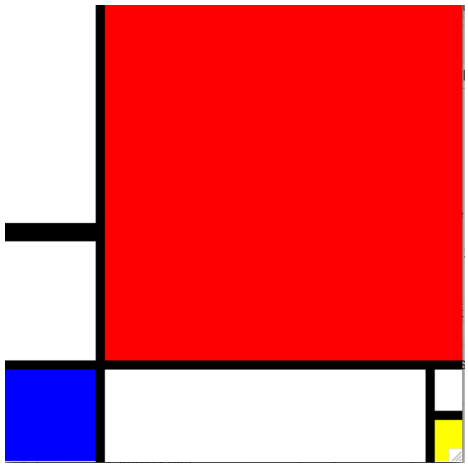

Figure 3: Representation of Mondrian's Composition II in Red, Blue, and Yellow

Instruction	Arguments	Description
`square`	`color x y w h`	Draws a filled rectangle at `(x, y)` with width `w` and height `h`.
`line`	`color lw x1 y1 x2 y2`	Draws a line from `(x1, y1)` to `(x2, y2)` with line width `lw`.
`image`	`file x y scale rotation`	Loads an image file and draws it at `(x, y)`, scaled by `scale` and rotated by `rotation` degrees.
`text`	`color x y size words...`	Draws a text string at `(x, y)` in the given font `size`. Everything after `size` is the text.
`call`	`label`	Executes the instructions of another label.
`translate`	`dx dy`	Shifts the drawing origin by `(dx, dy)` in the current frame, for all subsequent instructions in the label.
`rotate`	`deg`	Rotates the drawing frame by `deg` degrees, for all subsequent instructions in the label.

Instruction	Arguments	Description
`repeatplanar`	`label nx ny dx dy sy`	Draws `label` in a planar grid of `nx`×`ny` copies from left to right. Each copy in the x-direction is offset by `dx` from the previous on the left; each copy in the y-direction is offset by `dy` from the previous one below; additionally, each copy is offset by `(0, sy)` with relation to the previous on the left.
`repeatrotate`	`label n deg`	Draws `label n` times, rotating the drawing frame by `deg` degrees more on each copy, all around the current origin.

The coordinate system has `(0, 0)` in the lower-left corner of the canvas, with x increasing to the right and y increasing upward. Colours are: `black`, `white`, `red`, `blue`, `green`, `yellow`. Operations `translate` and `rotate` affect all instructions that follow them within the same label; they do not affect the caller after a `call` returns. Operations `repeatplanar` and `repeatrotate` execute the given label multiple times, applying the specified transformations to each copy.

3 Task

Analyse the code provided in the dune project and implement the required functions to extend the interpreter with the new operations. You should ensure that your implementation correctly handles the transformations and repetitions as specified, allowing users to create complex tiling patterns. An automatic testing procedure will be added when the autolab assignment is created, but you can also create your own test cases to verify the correctness of your implementation. You can use the provided examples as a starting point for your tests.

The code of the drawing primitives should not be modified (unless you need to cover for new cases of instructions). You should only add code to the `main.ml` file, and you can add helper functions as needed.

4 Execution of the interpreter

The interpreter can be executed with the following command:

```
dune exec ./main.exe <path_to_tiling_program>
```

Where `<path_to_tiling_program>` is the path to the file containing the tiling program you want to execute. You will see an image appearing in an X window and some output in the console. That sequence will be the testing data that evaluation will be based on. Look out for a new version that will include automatic testing and a more detailed output in the console.

5 Dependencies

The `image` instruction requires ImageMagick 7 to be installed.

macOS

```
brew install imagemagick
```

Linux / WSL (Ubuntu, Debian)

```
sudo apt install imagemagick
```

6 Submission details

The project will be submitted via autolab, and will be tested against a hidden test suite. You can run the public tests with `dune test`. The autolab assignment will be available at `https://lap.di.fct.unl.pt` soon. You can submit as many times as you want before the deadline, but only your last submission will be graded.

7 Rules

This assignment must be completed individually. You are allowed to discuss ideas with your classmates and ask questions to the instructors and in discord, but you are not allowed to share code with anyone else. Please refer to the course rules online and the NOVA FCT Knowledge Assessment Regulation for more details on what is considered fraud and plagiarism, and the consequences of such actions. Students who commit fraud on an assignment will not receive frequency for it.

8 Evaluation criteria

Your solution will be evaluated based on the following criteria:

- **Correctness**: your code should return the correct output for all test cases, including hidden ones. This will be the most important criterion, and will be worth 80% of the grade.
- **Code quality**: your code should be purely functional, well-structured, readable, and follow good programming practices. This includes using meaningful variable names, writing clear and concise code, and avoiding unnecessary complexity. This will be worth 20% of the grade.

9 Samples

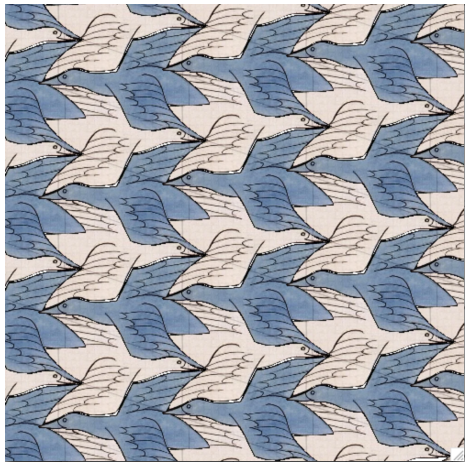

Figure 4: Sample 2

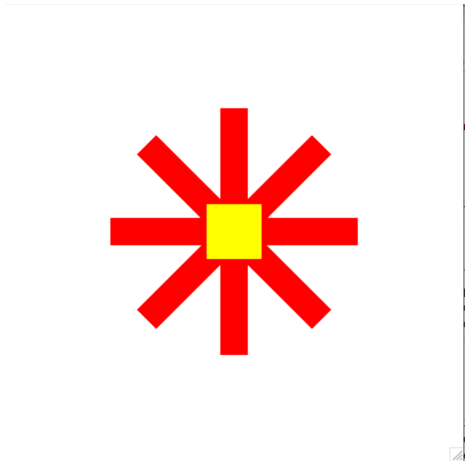

Figure 5: Sample 5